Normalization Layers

미래연구소 16기 11주차

12주차 내용 중 이론적으로 가장 어려운 부분입니다.

그러면서도 deep learning 분야에서 많이 쓰이는 기법 중 하나입니다.

큰 흐름 정도만 파악해 보셔도 충분합니다.

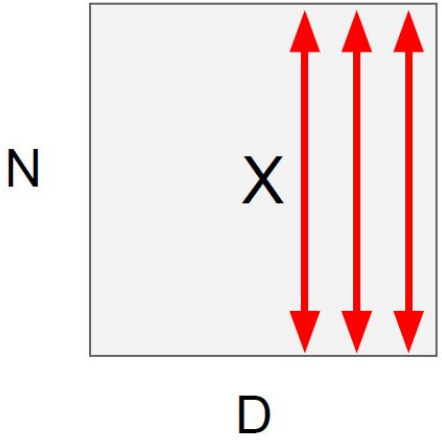
중요한 건 이를 여러분들이 만든 모델에 사용해보는 정도면 충분합니다.

1. Batch Normalization (복습) 1) 정의

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} (x_{i,j} - \mu_j)^2$$

Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

1. Batch Normalization (복습) 2) test?

Batch Normalization: Test-Time

Estimates depend on minibatch; can't do this at test-time!

Input: $x : N \times D$

Learnable scale and shift parameters:

$\gamma, \beta : D$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} (x_{i,j} - \mu_j)^2$$

Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

1. Batch Normalization (복습) 3) test - moving average

Batch Normalization: Test-Time

running average
= moving average

Input: $x : N \times D$

Learnable scale and shift parameters:

$\gamma, \beta : D$

During testing batchnorm becomes a linear operator!
Can be fused with the previous fully-connected or conv layer

$\mu_j =$ (Running) average of values seen during training — Per-channel mean, shape is D

$\sigma_j^2 =$ (Running) average of values seen during training — Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

1. Batch Normalization (복습) 4) shape

Batch Normalization

Batch Normalization for
fully-connected networks

$$x: N \times D$$

Normalize ↓

$$\mu, \sigma: 1 \times D$$

$$\gamma, \beta: 1 \times D$$

$$y = \gamma(x-\mu)/\sigma+\beta$$

N = batch_size (data 개수) 부분

D = feature

N을 기준으로 평균을 구한다.
(그래서 평균과 분산의 shape은 1XD이다.)

여기서부터는 심화 내용이며 큰 흐름 정도만 파악해도 괜찮습니다.

코드로 구현은 쉽습니다.

1. Batch Normalization (복습) 5) 다른 NN에 적용

Batch Normalization for ConvNets

Batch Normalization for
convolutional networks
(Spatial Batchnorm, BatchNorm2D)

$$x: \ N \times C \times H \times W$$

Normalize

$$\mu, \sigma: \ 1 \times C \times 1 \times 1$$

$$\gamma, \beta: \ 1 \times C \times 1 \times 1$$

$$y = \gamma(x - \mu) / \sigma + \beta$$

N = batch_size (data 개수) 부분

C = channel 수

H = height / W = weight

(RNN의 경우 H 혹은 W가 하나 없다고 생각하고 보시면 됩니다.)

1. Batch Normalization (복습) 6) FC와 CNN 비교

Batch Normalization for ConvNets

Batch Normalization for
fully-connected networks

$$x: N \times D$$

Normalize ↓

$$\mu, \sigma: 1 \times D$$

$$\gamma, \beta: 1 \times D$$

$$y = \gamma(x-\mu)/\sigma+\beta$$

Batch Normalization for
convolutional networks
(Spatial Batchnorm, BatchNorm2D)

$$x: N \times C \times H \times W$$

Normalize ↓ ↓ ↓

$$\mu, \sigma: 1 \times C \times 1 \times 1$$

$$\gamma, \beta: 1 \times C \times 1 \times 1$$

$$y = \gamma(x-\mu)/\sigma+\beta$$

```
[ ] model = tf.keras.models.Sequential([
      # Reshape into "channels last" setup.
      tf.keras.layers.Reshape((28,28,1), input_shape=(28,28)),
      tf.keras.layers.Conv2D(filters=10, kernel_size=(3,3),data_format="channels_last"),
      # LayerNorm Layer
      tf.keras.layers.BatchNormalization(),
      tf.keras.layers.Flatten(),
      tf.keras.layers.Dense(128, activation='relu'),
      tf.keras.layers.Dropout(0.2),
      tf.keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    model.fit(x_test, y_test)
```

```
Model: "sequential_3"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
reshape_3 (Reshape)          (None, 28, 28, 1)         0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 26, 26, 10)        100
_________________________________________________________________
batch_normalization (BatchNo (None, 26, 26, 10)        40
_________________________________________________________________
flatten_3 (Flatten)          (None, 6760)              0
_________________________________________________________________
dense_6 (Dense)              (None, 128)               865408
_________________________________________________________________
dropout_3 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_7 (Dense)              (None, 10)                1290
=================================================================
Total params: 866,838
Trainable params: 866,818
Non-trainable params: 20
```

1

It is clear that if you freeze any layer of the network. all params on that frozen layer turn to non-trainable. On the other hand if you design your network from the scratch, it might have some non-trainable parameters too. For instance batchnormalization layer has 4 parameter which are;

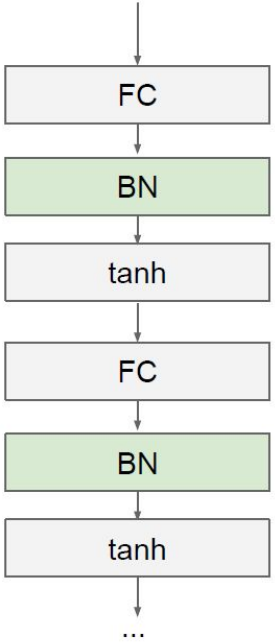
```
[gamma weights, beta weights, moving_mean, moving_variance]
```

The first two of them are trainable but last two are not. So the batch normalization layer is highly probably the reason that your custom network has non-trainable paramteres.

1. Batch Normalization (복습) 7) 효과

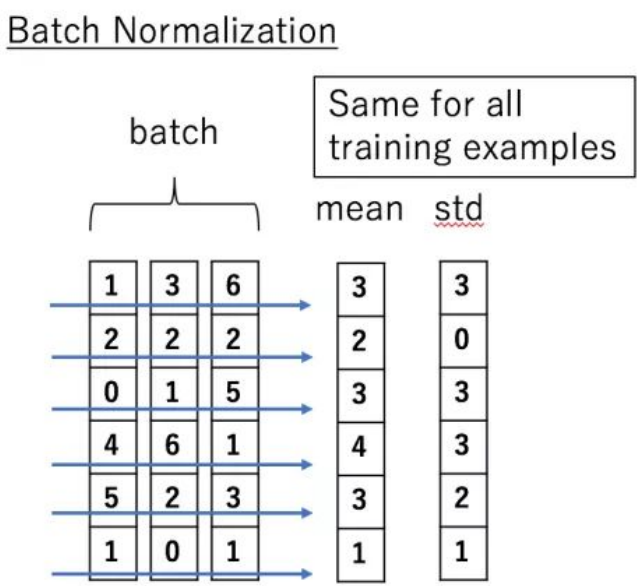
Batch Normalization

[Ioffe and Szegedy, 2015]



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!

1. Batch Normalization 8) 단점

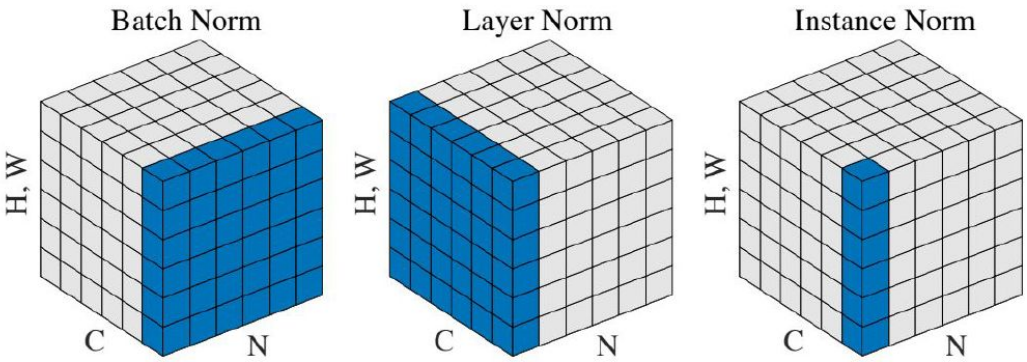


1> batch_size가 작으면 평균, 분산이 너무 부정확하다.
(특히, batch_size가 1이면 분산이 0이다.)

2> RNN은 같은 layer에서도 cell의 이동에 따라 다른 feature들을 받으므로 통계량이 매번 다르다.
(그렇다고 cell마다 BN을 적용하기에 너무 연산량이 많습니다.)

새로운 기법들의 등장

Comparison of Normalization Layers



2. Layer Normalization 1) Shape 비교

Layer Normalization

Batch Normalization for
fully-connected networks

$$x: N \times D$$

Normalize

$$\mu, \sigma: 1 \times D$$

$$\gamma, \beta: 1 \times D$$

$$y = \gamma(x-\mu)/\sigma+\beta$$

Layer Normalization for
fully-connected networks
Same behavior at train and test!
Can be used in recurrent networks

$$x: N \times D$$

Normalize

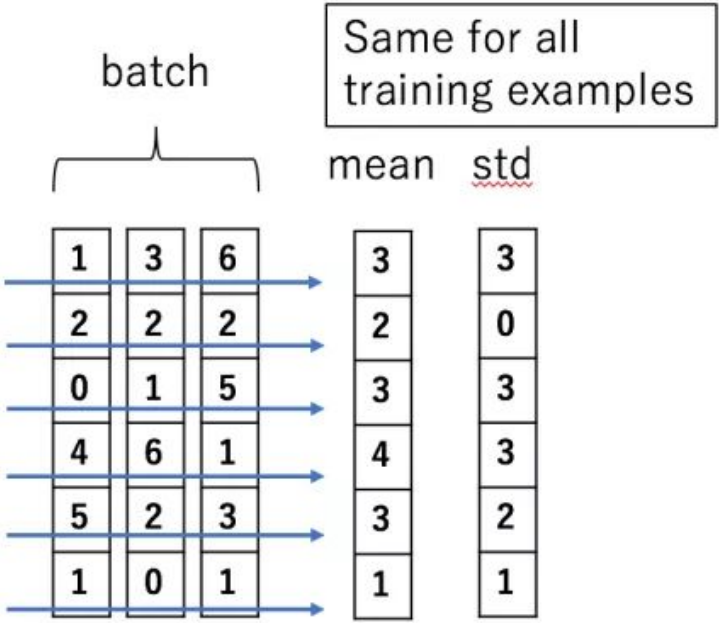
$$\mu, \sigma: N \times 1$$

$$\gamma, \beta: 1 \times D$$

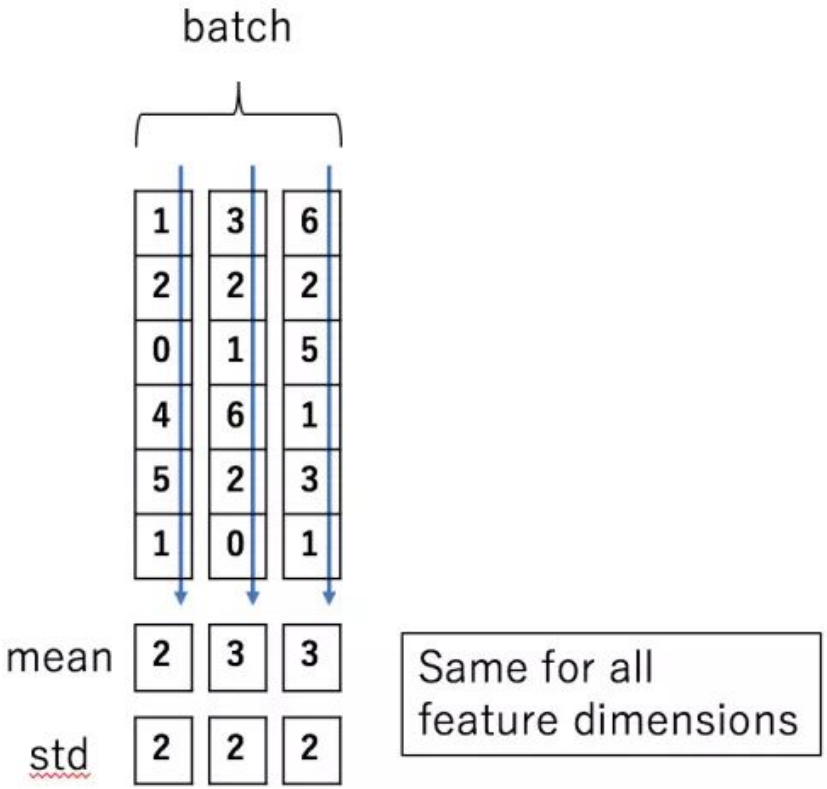
$$y = \gamma(x-\mu)/\sigma+\beta$$

Ba, Kiros, and Hinton, “Layer Normalization”, arXiv 2016

Batch Normalization



Layer Normalization



```python
model = tf.keras.models.Sequential([
  # Reshape into "channels last" setup.
  tf.keras.layers.Reshape((28,28,1), input_shape=(28,28)),
  tf.keras.layers.Conv2D(filters=10, kernel_size=(3,3),data_format="channels_last"),
  # LayerNorm Layer
  tf.keras.layers.LayerNormalization(axis=1 , center=True , scale=True),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_test, y_test)
```

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
reshape_2 (Reshape)          (None, 28, 28, 1)         0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 26, 26, 10)        100
_________________________________________________________________
layer_normalization (LayerNo (None, 26, 26, 10)        52
_________________________________________________________________
flatten_2 (Flatten)          (None, 6760)              0
_________________________________________________________________
dense_4 (Dense)              (None, 128)               865408
_________________________________________________________________
dropout_2 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_5 (Dense)              (None, 10)                1290
=================================================================
Total params: 866,850
Trainable params: 866,850
Non-trainable params: 0
```

3. Instance Normalization 1) Batch Normalization과의 비교

Instance Normalization

Batch Normalization for convolutional networks

x: N×C×H×W

Normalize

μ, σ: 1×C×1×1

γ, β: 1×C×1×1

$y = \gamma(x-\mu)/\sigma+\beta$

Instance Normalization for convolutional networks
Same behavior at train / test!

x: N×C×H×W

Normalize

μ, σ: N×C×1×1

γ, β: 1×C×1×1

$y = \gamma(x-\mu)/\sigma+\beta$

Ulyanov et al, Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis, CVPR 2017

```python
model = tf.keras.models.Sequential([
  # Reshape into "channels last" setup.
  tf.keras.layers.Reshape((28,28,1), input_shape=(28,28)),
  tf.keras.layers.Conv2D(filters=10, kernel_size=(3,3),data_format="channels_last"),
  # LayerNorm Layer
  tfa.layers.InstanceNormalization(axis=3,
                                   center=True,
                                   scale=True,
                                   beta_initializer="random_uniform",
                                   gamma_initializer="random_uniform"),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_test, y_test)
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
reshape_1 (Reshape)          (None, 28, 28, 1)         0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 26, 26, 10)        100
_________________________________________________________________
instance_normalization (Inst (None, 26, 26, 10)        20
_________________________________________________________________
flatten_1 (Flatten)          (None, 6760)              0
_________________________________________________________________
dense_2 (Dense)              (None, 128)               865408
_________________________________________________________________
dropout_1 (Dropout)          (None, 128)               0
_________________________________________________________________
dense_3 (Dense)              (None, 10)                1290
=================================================================
Total params: 866,818
Trainable params: 866,818
Non-trainable params: 0
```

자료 출처: http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture07.pdf